

String Indexing

What is String Indexing?

String Indexing comes in many forms but in all cases it boils down to pre-processing data to create new Machine Learning model feature(s) from raw string/categorical/text field(s). The generated features are always numerical – either scalars or vectors.

Pulse and the `string_indexing` library provide support for [Frequency Indexing](#). In some circles this is also known as **Count Indexing**.

There are two steps to use (frequency) indexers in Pulse:

1. **Fit** - Produce an indexer from train data using the `string_indexing` DS-API library:

```
indexer = string_indexing.fit(train_data, feature_name)
```

2. **Transform** - Transform input data using that indexer to add the new feature: Performed inside Pulse plans through the new `FrequencyIndexerUDF`.

About Frequency Indexing

Frequency Indexing is one of the simplest yet surprisingly effective forms of Indexing.

The pros of Frequency Indexing are its simplicity in terms of tweaking and robustness to over-fit when compared with other forms of indexing. Internal Feedzai benchmarks show recall gains of 8-15pp or 20-40%.

Say you have a string field with the country. Instead of defining the possible values of a categorical field, you can use Frequency Indexing.

The first step is to `fit` a new indexer with a train dataset that includes that field. This will:

1. Compute the number of occurrences of all the categories in that dataset.
2. Rank them in descending number of occurrences. The most frequent value will become 0, the second most frequent 1, and so on and so forth.
3. Store that mapping (string -> rank) and default output value for unknown input values.

What if you have values in the test dataset that you haven't seen before?

That's expected. Let's call those the "other" values. The Frequency Indexer will map those to the integer N, where N is the total number of categories stored in the indexer mapping.

This ensures the output value of the indexer is well defined even for unknown values.

Indexer clipping and good Data Science modeling practices

Is there a possibility your indexer will eventually receive new or unforeseen values?

If so, you should train your **Machine Learning model** by clipping your indexers first so that the indexed train dataset has "other" values that the indexer will convert to N. These train examples will make the model generalize better for new "other" values observed while predicting.

How to perform clipping^{AI}

Two forms of clipping are supported:

- Max categories (**M**): During the indexer fit step, only the **M** most frequent categories from the train data are kept in the indexer.
- Min samples per category (**S**): Any category/string seen less than **S** times during the indexer fit is dropped from the indexer.

These parameters can be tuned, but sensible values are already used by default, which work well for most scenarios.

Note

Although it can be tweaked, Pulse's `FrequencyIndexerUDF` will by default clip the indexer to **90%** of its input categories. This means that even if you forget to tweak **M** and **S**, the pre-processing pipeline already prevents over-fit and ensures that your Machine Learning model's train and evaluation data will contain "other" values.

Workflow^{AI}

The whole user journey when relying on indexing is:

1. Create model with Indexed features in the Data Science environment:
2. Create your Pulse indexer(s) using the `string_indexing` DS-API module.
3. Go to your Pulse Plan and using that Pulse Indexer and the new Pulse `FrequencyIndexerUDF` convert a raw field to an indexed feature.
4. Train a Machine Learning model with those features.
5. Put the model (and indexers) in production

To put the model in another environment, carry the model, features plan and the lists associated to the indexers. Indexers are currently stored as Pulse Lists whose name starts with prefix `"{StringIndexer}"`. Hence, follow the usual procedure to copy lists across environments.